Loyalty cards: 10-week build plan

6 October 2026

Two people have ten weeks, at about 8-10 hours a week each, to build a platform where any small business can sign up, make its own stamp card and have its customers add it to Apple Wallet. The plan follows the build order in the project notes, with the Wallet work moved earlier because it is the riskiest part.

Scope

The MVP is option 3 from the project notes, where staff scan the customer's card. It is stamps only and Apple Wallet only, but any business can sign up. It covers all 48 user stories in their simplest form.

Area	In the 10 weeks	After week 10
Businesses	Self-serve signup, email verification, logo, colours, stamp count, reward text, card preview, QR download	Stripe subscriptions and the pricing tiers, multiple locations
Customers	Join by QR with name and email, Apple Wallet card, web card fallback for Android	Google Wallet, customer login
Staff	Staff accounts, camera scanner, add stamp, redeem reward	Offline mode, till integration
Loyalty types	N stamps = one reward	Points per pound, visit counts, birthday rewards
Dashboard	Customer count, recent activity, rewards redeemed	Analytics, retention, most loyal customers
Marketing	Nothing	"We haven't seen you for 30 days" messages
Data protection	Privacy policy, consent on join, delete a customer	Data export

No story is dropped, but US-13 to US-16 are delivered as one simple dashboard page instead of separate views.

Timeline

Every week ends with one demonstrable result. If the week 2 pass opens on an iPhone, the project is viable. If week 7 slips, week 9 absorbs it. Weeks 2 and 7 carry the most risk.

Week	Starts	Person A: backend and Wallet	Person B: design and frontend	Milestone
1	5 Oct	Schema, Apple enrolment	Figma flows for all three users	Scope agreed
2	12 Oct	**Auth and the Wallet spike**	React scaffold, login pages	**Pass opens**
3	19 Oct	Programme API, deployment	Card builder with live preview	Live on HTTPS
4	26 Oct	Join API, QR codes	Join page, web card	QR join works
5	2 Nov	Real pass generation	Add to Apple Wallet flow	Card in Wallet

Week	Starts	Person A: backend and Wallet	Person B: design and frontend	Milestone
6	9 Nov	Stamp API with validation	Staff scanner page	Stamps work
7	16 Nov	**Pass update web service**	Staff accounts, error states	**Live updates**
8	23 Nov	Reward and redeem logic	Redeem screen, dashboard	Full loop
9	30 Nov	Security review, tests, buffer	Polish, phone testing, buffer	Pilots agreed
10	7 Dec	Pilot support and fixes	Pilot support and fixes	Pilot live

Working arrangement

Work is split by layer so the two people rarely block each other. Person A owns the backend and Wallet, and Person B owns design and the three frontends. The roles can be swapped, but each layer should have one owner.

	Person A	Person B
Owns	Flask API, PostgreSQL, Apple Wallet passes, deployment	Figma, React (business, customer, staff), QR scanner
Hardest weeks	2, 5, 7	3, 6
Reviews	Every frontend pull request	Every backend pull request

- **Stack:** React, Flask and PostgreSQL, as in the project notes. SQLAlchemy with Alembic handles migrations, and a pass-signing library is used instead of writing the signing by hand.
- **Repo:** one GitHub repo with `/frontend` and `/backend`, using the folder layout from section 16 of the project notes. Nothing is pushed to `main` without the other person's review.
- **API contract first:** each week starts by agreeing the request and response format for that week's endpoints. Person B builds against mock data until the real endpoint is ready.
- **Rhythm:** a 30-minute planning call on Monday and a demo on Sunday. The week's "done when" line is what gets demoed.
- **Costs:** the Apple Developer Program is $99 a year (the price in pounds is shown at enrolment), a domain is about £10 a year, and hosting can start on a free tier.
- **Dates:** Week 1 starts on Monday 5 October 2026 and week 10 ends on Sunday 13 December 2026.

Weeks 1-2: Foundations

By the end of week 2 a business can register and log in, and a hand-made Wallet pass opens on a real iPhone. That second result decides whether the project is viable.

Week 1: Scope, design and schema (5-11 Oct)

Together

- Mark each of the 48 user stories as must, should or later
- Set up the repo, folder layout and a shared task board
- Enrol in the Apple Developer Program on day one, because approval can take a few days
- Pick a product name and buy the domain

Person A

- Design the schema below and write it as SQLAlchemy models
- Set up local PostgreSQL and the first Alembic migration

Person B

- Make a Figma prototype of the three flows from the project notes: business (sign up to QR code), customer (scan to Wallet) and staff (scan to +1 stamp)
- Include the empty, error and success states for every screen

Table	Key columns	Notes
`businesses`	id, name, email, password_hash, email_verified, logo_url, colour	One row per shop
`staff`	id, business_id, email, password_hash, active	`active` handles US-18
`loyalty_programs`	id, business_id, slug, stamps_required, reward, icon	`slug` is the join URL, e.g. `/join/joes-coffee`
`customers`	id, name, email, consent_at	`consent_at` records the privacy consent
`loyalty_cards`	id, token, customer_id, program_id, stamps, reward_status	`token` is a long random string used in the QR code, never the id
`transactions`	id, card_id, business_id, staff_id, type, created_at	Type is STAMP or REDEEM
`wallet_devices`	id, card_id, device_library_id, push_token	Needed in week 7 for pass updates

The project notes list a separate `rewards` table. For the MVP, `reward_status` on the card plus REDEEM rows in `transactions` is enough.

Done when: the Figma prototype clicks through all three flows, the schema is migrated locally, and the Apple enrolment is submitted.

Week 2: Auth and the Wallet spike (12-18 Oct)

Person A

- Flask app skeleton with blueprints that match the `/backend` folders
- `POST /api/auth/register`: validate, hash the password and create the business (section 2 of the project notes)
- `POST /api/auth/login` and session or JWT handling
- Email verification link on signup
- A helper that scopes every query to the logged-in business, used from here on
- **Wallet spike:** create the Pass Type ID and certificate, then sign one hardcoded `.pkpass` and open it on an iPhone

Person B

- React project with routing for `/business`, `/join` and `/staff`
- Sign up, log in and verify-email pages connected to the real endpoints
- Shared components: buttons, form fields, error messages and layout

Stories: US-01, US-02, US-42.

Done when: a new business can register, verify its email and get to an empty dashboard, and the test pass shows up in Wallet. If the pass still isn't working by Sunday, week 3 goes on that before anything else.

Weeks 3-4: Business setup and customer joining

By the end of week 4 a shop owner can design a card on their own and a customer can join it by scanning a QR code. The site goes live on HTTPS in week 3, because the camera scanner and Wallet updates both need it later.

Week 3: Loyalty programme creation (19-25 Oct)

Person A

- `POST /api/loyalty-program`: stamps required, reward text, icon and card colour (section 3 of the project notes)
- `GET` and `PUT /api/loyalty-program/:id` so the owner can edit their scheme
- Logo upload with size and file-type limits, stored in object storage
- Generate a unique slug for each programme's join URL
- Deploy the backend, database and frontend to a host with HTTPS. Secrets and the Apple certificate go in environment variables, never in the repo

Person B

- "Create loyalty card" form: business name, logo, colour, number of stamps and reward
- Live card preview that updates as the owner types
- Edit-programme page
- Usability test: watch one non-technical person make a card without any help, then fix whatever confused them

Stories: US-03 to US-10.

Done when: on the live site, someone who has never seen it can make a branded scheme in under five minutes.

Week 4: Customer joining and QR codes (26 Oct-1 Nov)

Person A

- `GET /api/join/:slug`: public programme details for the join page
- `POST /api/customers`: validate, then create the customer and a loyalty card at 0 stamps with a random token (section 7 of the project notes)
- If the email already has a card for that programme, return the existing card instead of making a second one
- QR code image for the join URL, as a PNG and a print-ready PDF
- `DELETE /api/customers/:id` so a business can remove someone if they ask
- Rate limit the signup and join endpoints

Person B

- Join page at `/join/:slug` with the business branding, the rule ("free coffee after 10 purchases") and name and email fields
- Consent line and a link to a privacy policy page

- Web card page that shows stamps as filled and empty circles. This is the card for Android users
- Dashboard panel: "Your loyalty programme is ready", the QR code, and download and print buttons

Stories: US-11, US-12, US-29, US-30, US-31, US-33.

Done when: scanning the printed QR code with a phone creates a customer and shows their web card at 0 of 10.

Weeks 5-6: Wallet pass and stamping

By the end of week 6 a customer has a branded card in Apple Wallet and staff can add a stamp by scanning it. The card in Wallet does not refresh by itself yet; that is week 7.

Week 5: Generating the real pass (2-8 Nov)

Person A

- Turn the week 2 spike into `POST /api/wallet/generate`, building the pass from the card, programme and business rows (section 8 of the project notes)
- Pass contents: business name, logo, colour, customer name, "4 / 10 stamps", reward text, and a QR code with the card token in it
- One Pass Type ID for the whole platform, with a unique serial number for each card
- Put the web service URL and an authentication token in every pass now, so passes do not need re-issuing in week 7
- Resize uploaded logos to the icon and logo sizes Wallet expects

Person B

- "Join & Add to Apple Wallet" button on the join page, using Apple's official badge artwork
- Detect the device: iPhone gets the pass download and everything else goes to the web card
- Design the stamp strip image, which is the row of filled and empty stamps on the pass
- Test on at least two different iPhones

Stories: US-32, US-34, US-35, US-37.

Done when: scanning the QR code, entering a name and email and tapping once puts the shop's branded card in Wallet.

Week 6: Staff scanning and stamps (9-15 Nov)

Person A

- `POST /api/loyalty/stamp` that takes the card token
- The backend checks from section 11 of the project notes: the card exists, belongs to this business, is valid and can take another stamp
- Write a STAMP row to `transactions` in the same database transaction as the stamp increment
- Duplicate guard: refuse a second stamp on the same card within a short window, such as two minutes
- `GET /api/loyalty-card/:token` that returns the customer name, stamps and reward status

Person B

- Staff page at `/staff` with a camera QR scanner in the browser, using a library like html5-qrcode

- After a scan, show the customer name, their current stamps and a large "+ Add stamp" button
- Clear success and error states: invalid card, wrong business, stamped too recently
- Recent transactions list under the scanner
- Make everything reachable with one thumb on a phone, because staff will be using it in the middle of a rush

Stories: US-20 to US-25, US-43, US-44, US-47, US-48.

Done when: scanning a Wallet card on the staff page takes the stamp count from 4 to 5 in the database and on the web card, and a card from another business is refused.

Weeks 7-8: Wallet updates and rewards

By the end of week 8 the whole loop works: ten stamps, a reward, a redemption and a reset, with the Wallet card changing each time. Week 7 is the hardest week of the plan.

Week 7: Updating the pass (16-22 Nov)

The project notes call this "the technically interesting part". Apple Wallet does not accept a pushed card. The server tells the phone something has changed, and the phone then asks the server for the new pass.

Person A

- Register endpoint: `POST /v1/devices/{device}/registrations/{passType}/{serial}` saves the device and its push token to `wallet_devices`
- Unregister endpoint: `DELETE` on the same path, called when a customer removes the card
- Updated-passes endpoint: `GET /v1/devices/{device}/registrations/{passType}` returns the serial numbers that changed since a given time
- Latest-pass endpoint: `GET /v1/passes/{passType}/{serial}` returns the freshly signed pass
- Log endpoint: `POST /v1/log`. Wallet reports its errors here, so it is the main debugging tool
- Check the authentication token on every one of these requests
- After each stamp, send an empty push notification through Apple's push service to every device registered for that card

Person B

- Staff accounts: the owner adds a staff member by email and can remove their access (US-17, US-18)
- Staff login that goes straight to the scanner, with no access to settings or customer lists
- "Stamp added" confirmation that shows the new balance
- Help Person A test, because this week needs real phones and lots of scan-and-wait

Stories: US-17, US-18, US-19, US-36, US-46.

Done when: staff scan a card and, within about a minute, the card in Wallet shows 5 / 10 without the customer doing anything.

If it isn't working by Sunday: carry it into week 8 alongside rewards and stop at Wednesday of week 9. After that, launch with the web card as the live balance and note that the Wallet card only refreshes when it is re-added.

Week 8: Rewards and the dashboard (23-29 Nov)

Person A

- When stamps reach `stamps_required`, set `reward_status` to AVAILABLE (section 13 of the project notes)
- `POST /api/loyalty/redeem`: check a reward is available, write a REDEEM transaction and reset stamps to 0 (section 14)
- Make redeeming safe against double taps, so one reward can only be redeemed once
- The pass shows "Reward available" and sends a Wallet notification when the tenth stamp lands
- `GET /api/business/dashboard`: customer count, stamps today, rewards redeemed and the last 20 transactions

Person B

- Staff screen at 10 / 10 that shows "Reward available" and a "Redeem reward" button instead of "Add stamp"
- Redemption confirmation
- Business dashboard page from section 15 of the project notes: the four numbers, recent activity, and a customer list with a delete option

Stories: US-13 to US-16, US-26 to US-28, US-38 to US-41, US-45.

Done when: one card goes from 0 to 10, gets redeemed and goes back to 0, and Wallet, the staff page and the dashboard all agree at every step.

Weeks 9-10: Hardening and pilot

By the end of week 10 two or three real businesses of different kinds have set themselves up and stamped real customers. No new features are added in week 9; it is the buffer for anything that slipped.

Week 9: Hardening (30 Nov-6 Dec)

Person A

- Security pass: log in as business X and try to read, stamp and redeem business Y's cards directly through the API
- Automated tests for the stamp and redeem rules, including the duplicate guard and double redemption
- Turn on database backups
- Error logging so pilot failures are visible before the café reports them

Person B

- Run every flow on a small iPhone, a large iPhone and an Android phone
- Polish: loading states, wording, the empty dashboard, and a landing page that explains the product on one screen
- A one-page printable guide for staff: open the page, scan, tap
- Write the privacy policy and terms pages

Together

- Finish anything carried over from weeks 7 and 8
- Approach five local businesses of different kinds (café, barber, nail salon, bakery) to land two or three pilots

Done when: both people can run the whole loop three times with no bugs, and at least two pilots have said yes.

Week 10: Pilot (7-13 Dec)

Together

- Give each pilot business the signup link and watch them set up without helping, and write down every time they hesitate
- Print their QR code and be in the shop for the first hour of real use
- Fix blocking bugs the same day and log everything else
- Count, for each business, how many customers joined, how many stamps were given, and how often staff forgot to scan
- End-of-week review: what to keep, what to change, and which post-MVP item comes first

Done when: real customers have real cards, and there is a ranked list of what to build next.

Risks

Most of the risk is in the two Wallet weeks. The rest is ordinary web work.

Risk	When	Fallback
Apple Developer enrolment is slow	Weeks 1-2	Do auth and programme setup first and move the spike to week 3
The signed pass won't open and gives no error	Weeks 2 and 5	Use a maintained signing library and compare against Apple's sample passes
Pass updates don't arrive	Week 7	Read the `/v1/log` output. If still stuck by the middle of week 9, launch with the web card as the live balance
The camera scanner is unreliable on some phones	Week 6	Add a manual "enter card code" box next to the scanner
Staff forget to scan	Week 10	Put a sign on the counter that tells customers to ask for their stamp
One person falls behind	Any week	Week 9 is the buffer. Cut the dashboard down to counts before cutting anything in the stamp loop
No business agrees to a pilot	Weeks 9-10	Offer it free for six months and set it up with them

First three things after week 10: Google Wallet, since about half of UK phones are Android, then Stripe subscriptions for the pricing tiers in the project notes, then the analytics page.